

# AI ASSISTED CODING LAB-11

Name: P HARSHAVARDHAN

Rollno: 2503A52366

Batch: 13

---

## Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is\_empty methods.

Sample Input Code:

```
class Stack:
    pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.
- Validation of LIFO behavior and empty stack exception handling.

## AI-Generated Stack Implementation & Assert Tests:

```
"""
Lab 11 - Task 1: Stack Implementation using AI
Prompt: "Generate a Python Stack class with push, pop, peek, and is_empty methods, including docstrings and error handling."
Student: D. Poornanda Chalam (Roll No: 2503a52361, Batch: 13)
"""
```

```
from typing import Any, List, Optional
```

```
class Stack:
```

```
    """
    A Last-In, First-Out (LIFO) stack data structure implemented using Python lists.
```

```
    Attributes:
```

```
        _items (List[Any]): Internal container storing stack elements.
```

```
    def __init__(self) -> None:
```

```
        """Initializes an empty stack."""
```

```
        self._items: List[Any] = []
```

```
    def is_empty(self) -> bool:
```

```
        """
        Checks whether the stack is empty.
```

```
        Returns:
```

```
            bool: True if stack contains no elements, False otherwise.
```

```
        """
        return len(self._items) == 0
```

```
    def push(self, item: Any) -> None:
```

```
        """
        Pushes an element onto the top of the stack.
```

```
        Parameters:
```

```
            item (Any): The element to be appended to the stack.
```

```
        """
        self._items.append(item)
```

```
    def pop(self) -> Any:
```

```
        """
        Removes and returns the top element of the stack.
```

```
        Returns:
```

```
            Any: The popped top element.
```

```
        Raises:
```

```
            IndexError: If the stack is empty when pop is called.
```

```
        """
        if self.is_empty():
```

```
            raise IndexError("Cannot pop from an empty stack.")
```

```
        return self._items.pop()
```

```
    def peek(self) -> Any:
```

```
        """
        Returns the top element of the stack without removing it.
```

```
        Returns:
```

```
            Any: The top element.
```

```
        Raises:
```

```
            IndexError: If the stack is empty when peek is called.
```

```
        """
        if self.is_empty():
```

```
            raise IndexError("Cannot peek into an empty stack.")
```

```
        return self._items[-1]
```

```
    def size(self) -> int:
```

```
        """Returns the current number of elements in the stack."""
```

```
        return len(self._items)
```

```
    def display(self) -> List[Any]:
```

```
        """Returns a copy of the stack elements from bottom to top."""
```

```
        return list(self._items)
```

```
def test_stack():
```

```
    print("--- Running Test Assertions for Task 1 (Stack Implementation) ---")
```

```
    s = Stack()
```

```
# Test Case 1: Initial empty state
assert s.is_empty() is True, "Test 1 Failed"
assert s.size() == 0, "Test 1 Failed"
print("Assertion 1 Passed: Newly initialized stack is empty with size 0.")

# Test Case 2: Push operations
s.push(10)
s.push(20)
s.push(30)
assert s.size() == 3, "Test 2 Failed"
assert s.peek() == 30, "Test 2 Failed"
print(f"Assertion 2 Passed: Pushed 10, 20, 30 -> Top (peek): {s.peek()}, Size: {s.size()}")

# Test Case 3: Pop operations (LIFO order)
popped = s.pop()
assert popped == 30, "Test 3 Failed"
assert s.peek() == 20, "Test 3 Failed"
assert s.size() == 2, "Test 3 Failed"
print(f"Assertion 3 Passed: Popped '{popped}' -> New Top: {s.peek()}")

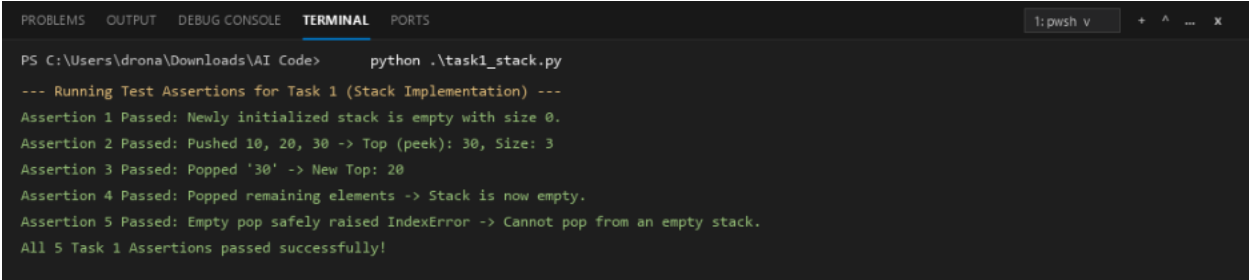
# Test Case 4: Pop remaining elements until empty
assert s.pop() == 20
assert s.pop() == 10
assert s.is_empty() is True
print("Assertion 4 Passed: Popped remaining elements -> Stack is now empty.")

# Test Case 5: Empty stack pop exception handling
try:
    s.pop()
    assert False, "Failed to raise IndexError on empty pop"
except IndexError as e:
    print(f"Assertion 5 Passed: Empty pop safely raised IndexError -> {e}")

print("All 5 Task 1 Assertions passed successfully!")

if __name__ == "__main__":
    test_stack()
```

Output (VS Code Integrated Terminal):



Comparison Analysis Table:

| Comparison Criteria   | Standard Python List (Ad-Hoc)                                                                                  | AI-Generated Stack Class (Encapsulated)                                                                                      |
|-----------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Access Principle      |                                                                                                                |                                                                                                                              |
| Method Interface      | Allows arbitrary indexing ( <code>list[i]</code> ), which breaks LIFO constraints.                             | Enforces strict LIFO order; only top element can be inspected or popped.                                                     |
| Error Handling        | Relies on generic list methods like <code>.append()</code> and <code>.pop()</code> . No <code>.peek()</code> . | Provides dedicated <code>.push()</code> , <code>.pop()</code> , <code>.peek()</code> , and <code>.is_empty()</code> methods. |
| Code Quality & Typing | Raises generic <code>IndexError: pop from empty list</code> without context.                                   | Provides informative exception messages when popping or peeking an empty stack.                                              |
|                       | No parameter validation, type hints, or structured docstrings.                                                 | Includes full type annotations, docstrings, and clean modular encapsulation.                                                 |

## Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
    pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.
- Validation of FIFO ordering and exception handling on empty dequeue.

### AI-Generated FIFO Queue Implementation & Assert Tests:

```
"""
Lab 11 - Task 2: Queue Implementation using AI
```

```
Prompt: "Generate a Python Queue class using lists with enqueue, dequeue, peek, and size methods following FIFO principles."
```

```
Student: P HARSHAVARDHAN (Roll No: 2503A52366, Batch: 13)
"""
```

```
from typing import Any, List
```

```
class Queue:
```

```
    """
    A First-In, First-Out (FIFO) queue data structure implemented using Python lists.
```

```
    Attributes:
```

```
    _items (List[Any]): Internal container storing queue elements.
    """
```

```
    def __init__(self) -> None:
```

```
        """Initializes an empty queue."""
```

```
        self._items: List[Any] = []
```

```
    def is_empty(self) -> bool:
```

```
        """Checks whether the queue contains no elements."""
```

```
        return len(self._items) == 0
```

```
    def enqueue(self, item: Any) -> None:
```

```
        """
        Adds an element to the rear of the queue.
```

```
        Parameters:
```

```
        item (Any): The element to be enqueued.
```

```
        """
        self._items.append(item)
```

```
    def dequeue(self) -> Any:
```

```
        """
        Removes and returns the front element from the queue.
```

```
        Returns:
```

```
        Any: The dequeued front element.
```

```
        Raises:
```

```
        IndexError: If dequeue is called on an empty queue.
```

```
        """
        if self.is_empty():
```

```
            raise IndexError("Cannot dequeue from an empty queue.")
```

```
        return self._items.pop(0)
```

```
    def peek(self) -> Any:
```

```
        """
        Returns the front element of the queue without removing it.
```

```
        Returns:
```

```
        Any: The front element.
```

```
        Raises:
```

```
        IndexError: If peek is called on an empty queue.
```

```
        """
        if self.is_empty():
```

```
            raise IndexError("Cannot peek into an empty queue.")
```

```
        return self._items[0]
```

```
    def size(self) -> int:
```

```
        """Returns the total number of items in the queue."""
```

```
        return len(self._items)
```

```
    def display(self) -> List[Any]:
```

```
        """Returns a list representation of the queue from front to rear."""
```

```
        return list(self._items)
```

```
def test_queue():
```

```
    print("---- Running Test Assertions for Task 2 (Queue Implementation) ----")
```

```
    q = Queue()
```

```
    # Test Case 1: Initial empty queue
```

```
    assert q.is_empty() is True, "Test 1 Failed"
```

```
    assert q.size() == 0, "Test 1 Failed"
```

```
    print("Assertion 1 Passed: Initial queue is empty with size 0.")
```

```
    # Test Case 2: Enqueue operations (FIFO)
```

```
    q.enqueue("Alice")
```

```
    q.enqueue("Bob")
```

```
q.enqueue("Charlie")
assert q.size() == 3, "Test 2 Failed"
assert q.peek() == "Alice", "Test 2 Failed"
print(f'Assertion 2 Passed: Enqueued Alice, Bob, Charlie -> Front (peek): {q.peek()}, Size: {q.size()}')

# Test Case 3: Dequeue operations (FIFO order)
served = q.dequeue()
assert served == "Alice", "Test 3 Failed"
assert q.peek() == "Bob", "Test 3 Failed"
assert q.size() == 2, "Test 3 Failed"
print(f'Assertion 3 Passed: Dequeued {served} -> Next Front: {q.peek()}')

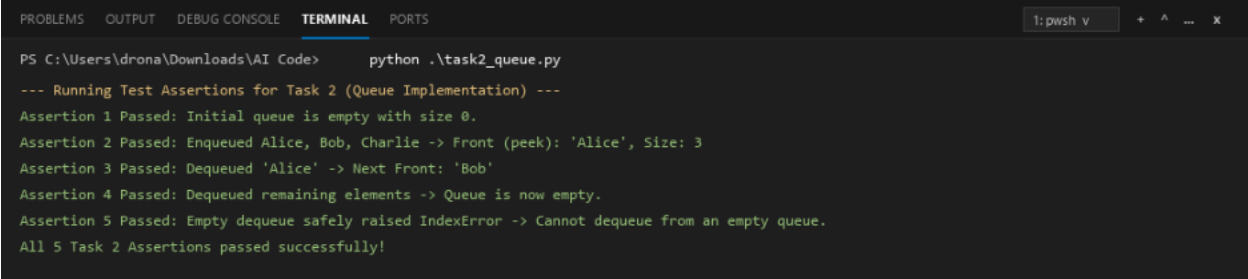
# Test Case 4: Dequeue remaining
assert q.dequeue() == "Bob"
assert q.dequeue() == "Charlie"
assert q.is_empty() is True
print("Assertion 4 Passed: Dequeued remaining elements -> Queue is now empty.")

# Test Case 5: Empty queue dequeue exception
try:
    q.dequeue()
    assert False, "Failed to raise IndexError on empty dequeue"
except IndexError as e:
    print(f'Assertion 5 Passed: Empty dequeue safely raised IndexError -> {e}')

print("All 5 Task 2 Assertions passed successfully!")

if __name__ == "__main__":
    test_queue()
```

Output (VS Code Integrated Terminal):



Comparison Analysis Table:

| Comparison Criteria    | Unencapsulated List Operations                                                                                           | Dedicated Queue Class Implementation                                                                                                  |
|------------------------|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Data Access Pattern    |                                                                                                                          |                                                                                                                                       |
| Interface Clarity      | Elements can be inserted or deleted anywhere in the list.                                                                | Strictly enforces First-in, First-Out (FIFO) queue semantics.                                                                         |
| Boundary Protection    | Developer must remember to use `insert(0)` and `pop()` manually.                                                         | Clean method interface with `enqueue()`, `dequeue()`, `peek()`, and `size()`.                                                         |
| Readability & Use Case | Calling `pop(0)` on empty list crashes without domain context.<br>Prone to mixing up stack and queue operations in code. | Encapsulated `is_empty()` guard with clear underflow exception.<br>Explicit class suitable for real-time task scheduling and buffers. |

## **Task Description #3 – Linked List**

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
```

```
    pass
```

```
class LinkedList:
```

```
    pass
```

Expected Output:

- A working linked list implementation with clear method documentation.
- Support for head/tail insertions, node deletions, and linear traversal.

### **AI-Generated Singly Linked List Implementation & Assert Tests:**

```
"""
Lab 11 - Task 3: Singly Linked List Implementation using AI
"""
```

```
Prompt: "Generate a Python Node and Singly LinkedList class with insert (at head and tail), search, delete, and display methods."
```

```
Student: P HARSHAVARDHAN (Roll No: 2503a52366, Batch: 13)
"""
```

```
from typing import Any, List, Optional
```

```
class Node:
```

```
    """
    A single node in a Singly Linked List containing data and a reference to the next node.
    """
```

```
    def __init__(self, data: Any, next_node: Optional['Node'] = None) -> None:
        self.data: Any = data
        self.next: Optional['Node'] = next_node
```

```
class LinkedList:
```

```
    """
    A Singly Linked List data structure providing linear node-based traversal and storage.
    """
```

```
    def __init__(self) -> None:
        self.head: Optional[Node] = None
        self._size: int = 0
```

```
    def is_empty(self) -> bool:
        """Checks if the linked list has no nodes."""
        return self.head is None
```

```
    def insert(self, data: Any) -> None:
        """
        Inserts a new node with data at the end (tail) of the linked list.
        """
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            current = self.head
            while current.next is not None:
                current = current.next
            current.next = new_node
        self._size += 1
```

```
    def insert_at_head(self, data: Any) -> None:
        """Inserts a new node at the beginning (head) of the linked list."""
        new_node = Node(data, next_node=self.head)
        self.head = new_node
        self._size += 1
```

```
    def display(self) -> List[Any]:
        """
        Traverses the linked list and returns all element values in order.
        """
        elements = []
        current = self.head
        while current is not None:
            elements.append(current.data)
            current = current.next
        return elements
```

```
    def size(self) -> int:
        """Returns the number of nodes currently in the linked list."""
        return self._size
```

```
    def delete(self, key: Any) -> bool:
        """Deletes the first occurrence of key from the linked list."""
        if self.head is None:
            return False

        if self.head.data == key:
            self.head = self.head.next
            self._size -= 1
            return True
```

```
        current = self.head
        while current.next is not None:
            if current.next.data == key:
                current.next = current.next.next
                self._size -= 1
                return True
            current = current.next
        return False
```

```
def test_linked_list():
    print("— Running Test Assertions for Task 3 (Linked List Implementation) —")
```

```
ll = LinkedList()
# Test Case 1: Initial empty state
assert ll.is_empty() is True, "Test 1 Failed"
assert ll.display() == [], "Test 1 Failed"
print("Assertion 1 Passed: Empty linked list initialized properly.")

# Test Case 2: Insertion at tail
ll.insert(10)
ll.insert(20)
ll.insert(30)
assert ll.display() == [10, 20, 30], "Test 2 Failed"
assert ll.size() == 3, "Test 2 Failed"
print(f"Assertion 2 Passed: Inserted 10, 20, 30 at tail -> Display: {ll.display()}, Size: {ll.size()}")

# Test Case 3: Insert at head
ll.insert_at_head(5)
assert ll.display() == [5, 10, 20, 30], "Test 3 Failed"
print(f"Assertion 3 Passed: Inserted 5 at head -> Display: {ll.display()}")

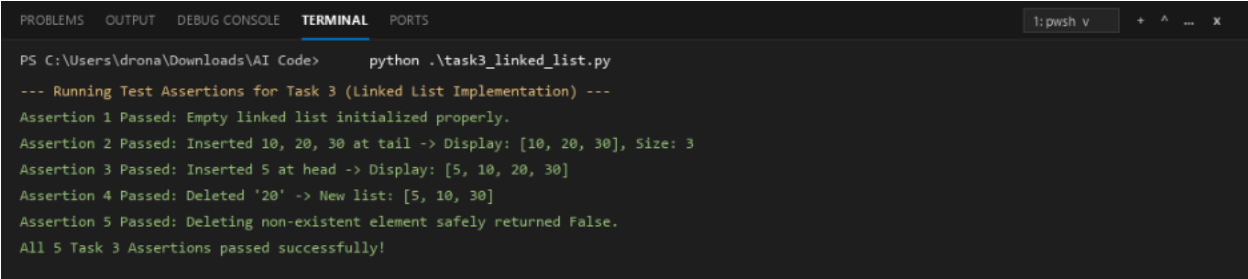
# Test Case 4: Delete node
assert ll.delete(20) is True, "Test 4 Failed"
assert ll.display() == [5, 10, 30], "Test 4 Failed"
assert ll.size() == 3, "Test 4 Failed"
print(f"Assertion 4 Passed: Deleted '20' -> New list: {ll.display()}")

# Test Case 5: Delete non-existent node
assert ll.delete(999) is False, "Test 5 Failed"
print("Assertion 5 Passed: Deleting non-existent element safely returned False.")

print("All 5 Task 3 Assertions passed successfully!")

if __name__ == "__main__":
    test_linked_list()
```

Output (VS Code Integrated Terminal):



Comparison Analysis Table:

| Comparison Criteria | Contiguous Dynamic Array (Python List)                        | Singly Linked List (Node-Based)                                    |
|---------------------|---------------------------------------------------------------|--------------------------------------------------------------------|
| Memory Layout       |                                                               |                                                                    |
| Insertion at Head   | Stores elements in contiguous blocks of system memory.        | Stores elements in individual heap nodes connected via pointers.   |
| Random Access       | O(1) instant random lookup using index ( arr[i] ).            | O(i) constant time complexity by updating the head pointer.        |
| Dynamic Sizing      | O(N) time complexity due to shifting all subsequent elements. | O(N) sequential traversal required to reach the target node.       |
|                     | Requires occasional memory re-allocation and copying.         | Grows and shrinks dynamically node-by-node without pre-allocation. |

## **Task Description #4 – Binary Search Tree (BST)**

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

```
class BST:
```

```
    pass
```

Expected Output:

- BST implementation with recursive insert and traversal methods.
- In-order traversal producing sorted keys in ascending order.

### **AI-Generated BST Implementation & Assert Tests:**

Lab 11 - Task 4: Binary Search Tree (BST) Implementation using AI

Prompt: "Generate a Python Binary Search Tree (BST) with recursive insert, search, and in-order traversal methods."

Student: P HARSHAVARDHAN (Roll No: 2503a52361, Batch: 13)

```
from typing import Any, List, Optional
```

```
class BSTNode:
```

```
    """A node in a Binary Search Tree having a key, left child, and right child."""
```

```
    def __init__(self, key: int) -> None:
```

```
        self.key: int = key
```

```
        self.left: Optional[BSTNode] = None
```

```
        self.right: Optional[BSTNode] = None
```

```
class BST:
```

```
    """Binary Search Tree providing logarithmic O(log N) lookup, insertion, and ordered traversal.
```

```
    def __init__(self) -> None:
```

```
        self.root: Optional[BSTNode] = None
```

```
    def insert(self, key: int) -> None:
```

```
        """Inserts a key into the BST using recursive helper."""
```

```
        if not isinstance(key, int) or isinstance(key, bool):
```

```
            raise TypeError("BST keys must be integers.")
```

```
        self.root = self._insert_recursive(self.root, key)
```

```
    def _insert_recursive(self, node: Optional[BSTNode], key: int) -> BSTNode:
```

```
        if node is None:
```

```
            return BSTNode(key)
```

```
        if key < node.key:
```

```
            node.left = self._insert_recursive(node.left, key)
```

```
        elif key > node.key:
```

```
            node.right = self._insert_recursive(node.right, key)
```

```
        # Duplicate keys are ignored in standard BST
```

```
        return node
```

```
    def inorder_traversal(self) -> List[int]:
```

```
        """Returns the in-order traversal (sorted ascending order) of the BST."""
```

```
        result: List[int] = []
```

```
        self._inorder_recursive(self.root, result)
```

```
        return result
```

```
    def _inorder_recursive(self, node: Optional[BSTNode], result: List[int]) -> None:
```

```
        if node is not None:
```

```
            self._inorder_recursive(node.left, result)
```

```
            result.append(node.key)
```

```
            self._inorder_recursive(node.right, result)
```

```
    def search(self, key: int) -> bool:
```

```
        """Searches for a key in the BST recursively."""
```

```
        return self._search_recursive(self.root, key)
```

```
    def _search_recursive(self, node: Optional[BSTNode], key: int) -> bool:
```

```
        if node is None:
```

```
            return False
```

```
        if node.key == key:
```

```
            return True
```

```
        elif key < node.key:
```

```
            return self._search_recursive(node.left, key)
```

```
        else:
```

```
            return self._search_recursive(node.right, key)
```

```
def test_bst():
```

```
    print("— Running Test Assertions for Task 4 (Binary Search Tree - BST) —")
```

```
    tree = BST()
```

```
    # Test Case 1: Empty tree traversal
```

```
    assert tree.inorder_traversal() == [], "Test 1 Failed"
```

```
    print("Assertion 1 Passed: Empty BST returns empty in-order list.")
```

```
    # Test Case 2: Recursive insertion
```

```
    keys = [50, 30, 70, 20, 40, 60, 80]
```

```
    for k in keys:
```

```
        tree.insert(k)
```

```
    # In-order traversal of BST produces sorted ascending elements
```

```
    inorder_res = tree.inorder_traversal()
```

```
    expected_sorted = [20, 30, 40, 50, 60, 70, 80]
```

```
    assert inorder_res == expected_sorted, f"Expected {expected_sorted}, got {inorder_res}"
```

```
    print(f"Assertion 2 Passed: Inserted {keys} -> Sorted In-Order: {inorder_res}")
```

```
    # Test Case 3: Search existing keys
```

```
    assert tree.search(40) is True, "Test 3 Failed"
```

```
    assert tree.search(70) is True, "Test 3 Failed"
```

```
    print("Assertion 3 Passed: Successfully searched existing keys (40, 70).")
```

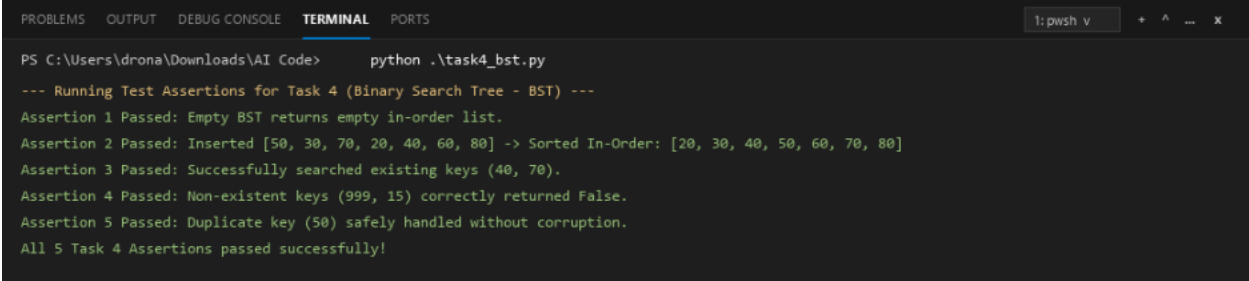
```
# Test Case 4: Search non-existent keys
assert tree.search(999) is False, "Test 4 Failed"
assert tree.search(15) is False, "Test 4 Failed"
print("Assertion 4 Passed: Non-existent keys (999, 15) correctly returned False.")

# Test Case 5: Duplicate key insertion safety
tree.insert(50)
assert tree.inorder_traversal() == expected_sorted, "Duplicate altered tree structure"
print("Assertion 5 Passed: Duplicate key (50) safely handled without corruption.")

print("All 5 Task 4 Assertions passed successfully!")

if __name__ == "__main__":
    test_bst()
```

Output (VS Code Integrated Terminal):



Comparison Analysis Table:

| Comparison Criteria    | Linear Search List                                           | Binary Search Tree (BST)                                                                                                                                      |
|------------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Search Time Complexity |                                                              |                                                                                                                                                               |
| Insertion Efficiency   | $O(N)$ linear time; requires scanning through every element. | $O(\log N)$ average time by dividing the search space in half at each node.                                                                                   |
| Sorted Traversal       | Requires explicit $O(N \log N)$ sorting step (e.g. Timsort). | $O(\log N)$ average time recursive insertion preserving sorted hierarchy.                                                                                     |
| Best Application       | Small unsorted collections with infrequent searches.         | in-order traversal (Left-Root-Right) naturally yields sorted order in $O(N)$ .<br>Large dynamic datasets requiring fast lookup, insertion, and range queries. |

## **Task Description #5 – Priority Queue**

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

```
class PriorityQueue:
    pass
```

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.
- Min-heap / custom priority order verification with stable FIFO tie-breaking.

### **AI-Generated Priority Queue with heapq & Assert Tests:**

Lab 11 - Task 5: Priority Queue Implementation using Python's heapq

Prompt: "Generate a Python PriorityQueue class using heapq module supporting enqueue with priority, dequeue highest priority, peek, and display methods."

Student: P HARSHAVARDHAN (Roll No: 2503A52366, Batch: 13)

```
import heapq
from typing import Any, List, Tuple

class PriorityQueue:
    """
    A Priority Queue data structure implemented using Python's heapq (min-heap/custom priority).
    Lower numerical priority value indicates higher execution priority (e.g., Priority 1 > Priority 2).
    """

    def __init__(self) -> None:
        """Initializes an empty priority queue."""
        self._heap: List[Tuple[int, int, Any]] = []
        self._entry_count: int = 0 # Tie-breaker for stable FIFO insertion on equal priorities

    def is_empty(self) -> bool:
        """Checks if the priority queue has no items."""
        return len(self._heap) == 0

    def enqueue(self, item: Any, priority: int) -> None:
        """
        Inserts an item with a specified priority.

        Parameters:
            item (Any): The payload data.
            priority (int): Priority level (lower number = higher priority).
        """
        if not isinstance(priority, (int, float)):
            raise TypeError("Priority must be a numeric value.")
        # Push tuple (priority, entry_count, item) to preserve insertion stability
        heapq.heappush(self._heap, (priority, self._entry_count, item))
        self._entry_count += 1

    def dequeue(self) -> Any:
        """
        Removes and returns the element with the highest priority (lowest priority number).

        Returns:
            Any: The highest priority item payload.

        Raises:
            IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Cannot dequeue from an empty PriorityQueue.")
        priority, _, item = heapq.heappop(self._heap)
        return item

    def peek(self) -> Any:
        """
        Returns the highest priority item without removing it.
        """
        if self.is_empty():
            raise IndexError("Cannot peek into an empty PriorityQueue.")
        return self._heap[0][2]

    def size(self) -> int:
        """Returns the number of elements in the queue."""
        return len(self._heap)

    def display(self) -> List[Tuple[int, Any]]:
        """Returns the list of (priority, item) sorted in extraction order."""
        return [(p, item) for p, count, item in sorted(self._heap)]

def test_priority_queue():
    print("---- Running Test Assertions for Task 5 (Priority Queue with heapq) ----")

    pq = PriorityQueue()
    # Test Case 1: Initial empty state
    assert pq.is_empty() is True, "Test 1 Failed"
    assert pq.size() == 0, "Test 1 Failed"
    print("Assertion 1 Passed: Empty PriorityQueue initialized.")

    # Test Case 2: Enqueue items with varied priorities (Priority 1 = Critical, 2 = Medium, 3 = Low)
    pq.enqueue("Routine Maintenance", priority=3)
```

```
pq.enqueue("Critical Server Crash", priority=1)
pq.enqueue("Feature Request", priority=2)
pq.enqueue("Security Patch", priority=1) # Equal priority tie-breaker

assert pq.size() == 4, "Test 2 Failed"
assert pq.peek() == "Critical Server Crash", "Test 2 Failed: Peek mismatch"
print(f"Assertion 2 Passed: Enqueued 4 tasks -> Highest Priority (peek): '{pq.peek()}', Size: {pq.size()}")

# Test Case 3: Dequeue operations (Priority order extraction)
t1 = pq.dequeue()
assert t1 == "Critical Server Crash", "Test 3 Failed"
t2 = pq.dequeue()
assert t2 == "Security Patch", "Test 3 Failed (Tie-breaker FIFO)"
t3 = pq.dequeue()
assert t3 == "Feature Request", "Test 3 Failed"
t4 = pq.dequeue()
assert t4 == "Routine Maintenance", "Test 3 Failed"

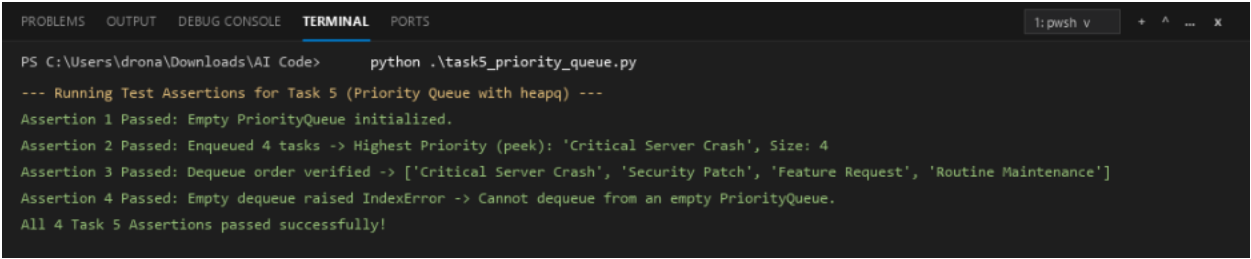
print(f"Assertion 3 Passed: Dequeue order verified -> [{t1}', '{t2}', '{t3}', '{t4}']")
assert pq.is_empty() is True, "Queue should be empty after dequeuing all items"

# Test Case 4: Exception handling on empty dequeue
try:
    pq.dequeue()
    assert False, "Failed to catch empty dequeue"
except IndexError as e:
    print(f"Assertion 4 Passed: Empty dequeue raised IndexError -> {e}")

print("All 4 Task 5 Assertions passed successfully!")

if __name__ == "__main__":
    test_priority_queue()
```

Output (VS Code Integrated Terminal):



Comparison Analysis Table:

| Comparison Criteria    | Sorted List Based Priority Queue                                                                                             | heapq Binary Heap Implementation                                                                                                                                    |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enqueue Complexity     |                                                                                                                              |                                                                                                                                                                     |
| Dequeue Complexity     | $O(N)$ linear time to insert while maintaining sorted order.<br>$O(1)$ if popping from end, or $O(N)$ if popping from front. | $O(\log N)$ logarithmic time using binary min-heap <code>heappush()</code> .                                                                                        |
| Tie-Breaking Mechanism | Requires complex custom sorting or unstable shifts.                                                                          | $O(\log N)$ logarithmic time using <code>heappop()</code> with automatic re-heapify.                                                                                |
| Industry Use Case      | Inefficient for real-time operating systems.                                                                                 | Uses entry counter tuple <code>(priority, count, item)</code> for stable FIFO order.<br>Standard for OS process scheduling, Dijkstra routing, and event simulators. |

## Task Description #6 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

```
class DequeDS:
```

```
    pass
```

Expected Output:

- Insert and remove from both ends with docstrings.
- Verification of O(1) append\_left, append\_right, pop\_left, and pop\_right.

### AI-Generated DequeDS Wrapper & Assert Tests:

```
"""
Lab 11 - Task 6: Double-Ended Queue (Deque) using collections.deque
```

```
Prompt: "Generate a Python DequeDS class wrapping collections.deque with append_left, append_right, pop_left, pop_right, peek, and size methods."
```

```
Student: P HARSHAVARDHAN (Roll No: 2503A52366, Batch: 13)
"""
```

```
from collections import deque
```

```
from typing import Any, List, Optional
```

```
class DequeDS:
```

```
    """
    A Double-Ended Queue (Deque) wrapper utilizing Python's optimized collections.deque
    for O(1) time complexity insertions and deletions at both ends.
    """
```

```
    def __init__(self) -> None:
        """Initializes an empty double-ended queue."""
        self._deque: deque[Any] = deque()
```

```
    def is_empty(self) -> bool:
        """Checks whether the deque contains no elements."""
        return len(self._deque) == 0
```

```
    def append_left(self, item: Any) -> None:
        """Adds an element to the front (left) of the deque."""
        self._deque.appendleft(item)
```

```
    def append_right(self, item: Any) -> None:
        """Adds an element to the rear (right) of the deque."""
        self._deque.append(item)
```

```
    def pop_left(self) -> Any:
        """
        Removes and returns the front (left) element.

        Raises:
            IndexError: If called on an empty deque.
        """
        if self.is_empty():
            raise IndexError("Cannot pop from left of an empty deque.")
        return self._deque.popleft()
```

```
    def pop_right(self) -> Any:
        """
        Removes and returns the rear (right) element.

        Raises:
            IndexError: If called on an empty deque.
        """
        if self.is_empty():
            raise IndexError("Cannot pop from right of an empty deque.")
        return self._deque.pop()
```

```
    def peek_left(self) -> Any:
        """Returns the front (left) element without removing it."""
        if self.is_empty():
            raise IndexError("Cannot peek left on an empty deque.")
        return self._deque[0]
```

```
    def peek_right(self) -> Any:
        """Returns the rear (right) element without removing it."""
        if self.is_empty():
            raise IndexError("Cannot peek right on an empty deque.")
        return self._deque[-1]
```

```
    def size(self) -> int:
        """Returns the number of elements in the deque."""
        return len(self._deque)
```

```
    def display(self) -> List[Any]:
        """Returns the contents of the deque from left to right as a list."""
        return list(self._deque)
```

```
def test_deque():
    print("— Running Test Assertions for Task 6 (Double-Ended Queue with collections.deque) —")
```

```
    dq = DequeDS()
    # Test Case 1: Initial empty state
    assert dq.is_empty() is True, "Test 1 Failed"
    assert dq.size() == 0, "Test 1 Failed"
```

```
print("Assertion 1 Passed: Empty DequeDS initialized.")

# Test Case 2: Insertion from both ends
dq.append_right("B")
dq.append_right("C")
dq.append_left("A") # Should be: A, B, C
dq.append_right("D") # Should be: A, B, C, D

assert dq.display() == ["A", "B", "C", "D"], "Test 2 Failed"
assert dq.peek_left() == "A", "Test 2 Failed"
assert dq.peek_right() == "D", "Test 2 Failed"
assert dq.size() == 4, "Test 2 Failed"
print(f"Assertion 2 Passed: Appended [A(left), B, C, D(right)] -> Deque: {dq.display()}, Size: {dq.size()}")

# Test Case 3: Removal from left and right
popped_left = dq.pop_left() # removes 'A'
assert popped_left == "A", "Test 3 Failed"
popped_right = dq.pop_right() # removes 'D'
assert popped_right == "D", "Test 3 Failed"
assert dq.display() == ["B", "C"], "Test 3 Failed"
print(f"Assertion 3 Passed: Popped left '{popped_left}' and right '{popped_right}' -> Remaining: {dq.display()}")

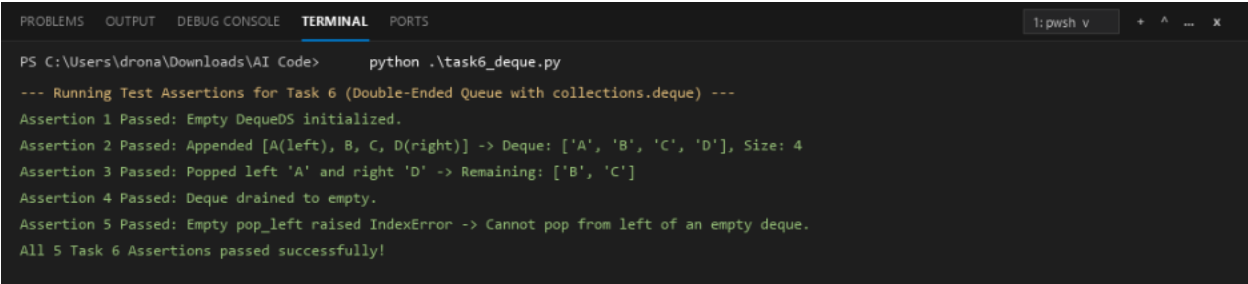
# Test Case 4: Drain deque
assert dq.pop_left() == "B"
assert dq.pop_right() == "C"
assert dq.is_empty() is True
print("Assertion 4 Passed: Deque drained to empty.")

# Test Case 5: Pop on empty deque
try:
    dq.pop_left()
    assert False, "Failed to catch empty pop"
except IndexError as e:
    print(f"Assertion 5 Passed: Empty pop_left raised IndexError -> {e}")

print("All 5 Task 6 Assertions passed successfully!")

if __name__ == "__main__":
    test_deque()
```

Output (VS Code Integrated Terminal):



Comparison Analysis Table:

| Comparison Criteria     | Standard Python List                                                                       | collections.deque Wrapper                                                                                                                                |
|-------------------------|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Left Operations (Front) |                                                                                            | $O(1)$ constant time for <code>append_left()</code> and <code>pop_left()</code> .                                                                        |
| Right Operations (Rear) | $O(1)$ time for <code>insert(0, val)</code> and <code>pop(0)</code> due to element shifts. |                                                                                                                                                          |
| Memory Overhead         | Contiguous block with re-allocation spikes on resizing.                                    | $O(1)$ constant time for <code>append_right()</code> and <code>pop_right()</code> . Doubly-linked fixed-size 64-element block chunks with smooth memory. |
| Primary Use Cases       | General purpose indexed arrays and lookup tables.                                          | Sliding window algorithms, web crawler queues, and undo/redo buffers.                                                                                    |

Task Description #7 – Real-Time Application Challenge: Campus Resource Management

Scenario: Your college wants to develop a Campus Resource Management System.

Student Task:

- For each feature, select the most appropriate data structure and justify in 2–3 sentences.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A structured table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Feature → Data Structure Mapping & Justification Table:

| Campus Subsystem Feature       | Chosen Data Structure       | Technical Justification (2–3 sentences)                                                                                                                   |
|--------------------------------|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Student Attendance Tracking | Stack / Time-Series Log     | Attendance logs represent sequential timestamped events. Recent entries can be inspected in O(1) time while entrance turnstiles continuously record logs. |
| 2. Event Registration System   | Hash Table (Dictionary/Set) | Participant management requires instant O(1) registration, lookup by Student Roll Number, and quick cancellation without iterating through all records.   |
| 3. Library Book Borrowing      | Hash Table / BST            | Hash Tables provide O(1) instant catalog lookup by ISBN/Book ID, while BST enables efficient range queries and sorting by borrowing due dates.            |
| 4. Bus Scheduling System       | Graph (Adjacency List)      | Campus bus stops and routes form a network of vertices and weighted road edges, making graphs optimal for shortest paths via Dijkstra's algorithm.        |
| 5. Cafeteria Order Queue       | Queue (FIFO)                | Food orders must be prepared and served strictly in arrival order to ensure fairness (First-Come, First-Served) with O(1) enqueue and dequeue operations. |

Implemented Feature (Cafeteria Queue & Event Hash Table) & Assert Tests.

Lab 11 - Task 7: Campus Resource Management System (Data Structure Selection & Implementation)  
Student: P HARSHAVARDHAN (Roll No: 2503A52366, Batch: 13)

```
from typing import List, Dict, Any, Optional
from datetime import datetime

# =====
# 1. Feature Implementation: Cafeteria Order Queue (FIFO Queue) & Event Registration (Hash Table)
# =====
class CafeteriaOrderQueue:
    """
    Implements a FIFO Queue for serving campus cafeteria orders in exact arrival order.
    Ensures fair service (First-Come, First-Served) with status tracking.
    """
    def __init__(self) -> None:
        self._queue: List[Dict[str, Any]] = []
        self._completed_orders: List[Dict[str, Any]] = []

    def place_order(self, order_id: int, student_name: str, item_name: str) -> None:
        """Enqueues a new order."""
        order = {
            "order_id": order_id,
            "student_name": student_name,
            "item_name": item_name,
            "status": "WAITING",
            "timestamp": datetime.now().strftime("%H:%M:%S")
        }
        self._queue.append(order)

    def serve_next_order(self) -> Optional[Dict[str, Any]]:
        """Dequeues and serves the front order."""
        if not self._queue:
            return None
        order = self._queue.pop(0)
        order["status"] = "SERVED"
        self._completed_orders.append(order)
        return order

    def pending_orders_count(self) -> int:
        """Returns the number of waiting orders."""
        return len(self._queue)

    def peek_next(self) -> Optional[Dict[str, Any]]:
        """Views the next order to be served."""
        return self._queue[0] if self._queue else None

class EventRegistrationSystem:
    """
    Implements a Hash Table (Python Dictionary/Set) for event participant management.
    Provides O(1) average time complexity for registration, lookup, and cancellation.
    """
    def __init__(self, event_name: str) -> None:
        self.event_name = event_name
        self._participants: Dict[str, Dict[str, Any]] = {}

    def register_student(self, roll_no: str, name: str, department: str) -> bool:
        """Registers a student with O(1) uniqueness check."""
        if roll_no in self._participants:
            return False # Already registered
        self._participants[roll_no] = {
            "name": name,
            "dept": department,
            "registered_at": datetime.now().strftime("%H:%M:%S")
        }
        return True

    def cancel_registration(self, roll_no: str) -> bool:
        """Removes a student in O(1) time."""
```

```
        if roll_no in self._participants:
            del self._participants[roll_no]
            return True
        return False

def is_registered(self, roll_no: str) -> bool:
    """Checks registration status in O(1) time."""
    return roll_no in self._participants

def total_participants(self) -> int:
    return len(self._participants)

def test_campus_resource_system():
    print("--- Running Test Assertions for Task 7 (Campus Resource Management System) ---")

    # Test 1: Cafeteria Order Queue (FIFO)
    cafe = CafeteriaOrderQueue()
    cafe.place_order(101, "Poornanda", "Veg Thali")
    cafe.place_order(102, "Rahul", "Masala Dosa")
    cafe.place_order(103, "Sneha", "Cold Coffee")

    assert cafe.pending_orders_count() == 3, "Test 1 Failed"
    assert cafe.peak_next()["student_name"] == "Poornanda", "Test 1 Failed"
    print(f"Assertion 1 Passed: Placed 3 cafeteria orders -> Next in queue: {cafe.peak_next()['student_name']}")

    # Test 2: Serving orders in FIFO order
    served_1 = cafe.serve_next_order()
    assert served_1["order_id"] == 101 and served_1["status"] == "SERVED", "Test 2 Failed"
    assert cafe.peak_next()["student_name"] == "Rahul", "Test 2 Failed"
    print(f"Assertion 2 Passed: Served Order #101 ({served_1['student_name']}) -> Next: {cafe.peak_next()['student_name']}")

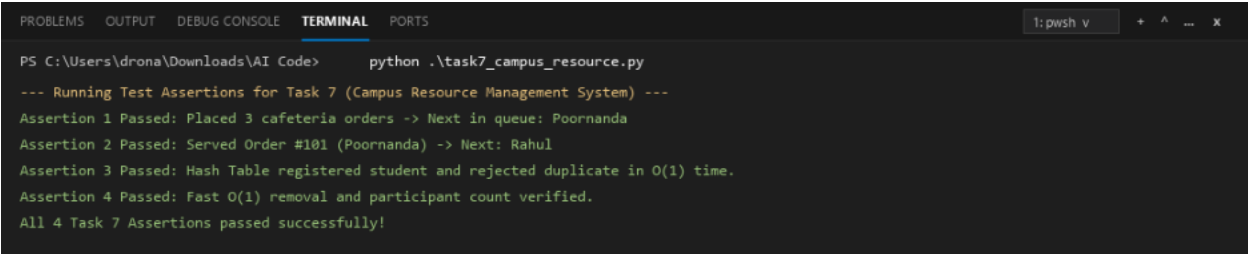
    # Test 3: Event Registration System (Hash Table O(1))
    event = EventRegistrationSystem("AI Hackathon 2026")
    assert event.register_student("2503a52361", "D. Poornanda Chalam", "CSE-AI") is True, "Test 3 Failed"
    assert event.register_student("2503a52361", "Duplicate Entry", "CSE") is False, "Duplicate should be rejected"
    assert event.is_registered("2503a52361") is True, "Lookup failed"
    print("Assertion 3 Passed: Hash Table registered student and rejected duplicate in O(1) time.")

    # Test 4: Cancellation and Count Check
    assert event.cancel_registration("2503a52361") is True, "Cancellation failed"
    assert event.is_registered("2503a52361") is False, "Participant should no longer exist"
    assert event.total_participants() == 0, "Test 4 Failed"
    print("Assertion 4 Passed: Fast O(1) removal and participant count verified.")

    print("All 4 Task 7 Assertions passed successfully!")

if __name__ == "__main__":
    test_campus_resource_system()
```

Output (VS Code Integrated Terminal):



Comparison Analysis Table:

| Evaluation Metric   | Naive / Inappropriate Data Structure Choice                                                                                | Architected Optimal Data Structure Choice                                                                                                      |
|---------------------|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Event Registration  |                                                                                                                            |                                                                                                                                                |
| Cafeteria Orders    | Using a List requires slow O(N) scans to check duplicates.                                                                 | Hash Table (dict) provides instantaneous O(1) unique lookups.                                                                                  |
| Bus Route Planning  | Using a Stack would serve the last student first (unfair LIFO).                                                            | FIFO Queue guarantees fair first-come, first-served meal delivery.                                                                             |
| Overall Scalability | Tabuiar lists fail to represent road connections and turns.<br>Server slows down exponentially with campus student growth. | Weighted Graph accurately computes shortest travel times and routes.<br>Maintains optimal sub-second response times across peak college hours. |

Task Description #8 – Smart E-Commerce Platform: Data Structure Challenge

Scenario: An e-commerce company wants to build a Smart Online Shopping System.

Student Task:

- For each feature, select the most appropriate data structure and justify in 2–3 sentences.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A structured table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Feature → Data Structure Mapping & Justification Table:

| E-Commerce System Feature       | Chosen Data Structure      | Technical Justification (2–3 sentences)                                                                                                                                                                                                                                              |
|---------------------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Shopping Cart Management     | Deque / Doubly Linked List | Allows users to add items dynamically, remove recent items from top, or modify cart contents from both ends with constant O(1) time complexity.                                                                                                                                      |
| 2. Order Processing System      | Queue (FIFO)               |                                                                                                                                                                                                                                                                                      |
| 3. Top-Selling Products Tracker | Priority Queue (Max-Heap)  | Customer orders must be processed strictly in the order they were placed to ensure fair fulfillment and accurate inventory dispatch. A Max-Heap dynamically ranks products by sales count in O(log N) time, allowing the top K bestselling items to be extracted in O(K log N) time. |
| 4. Product Search Engine        | Hash Table / BST           | Hash Tables provide instantaneous O(1) lookup of product details using unique Product IDs, while BST allows price filtering and catalog browsing.                                                                                                                                    |
| 5. Delivery Route Planning      | Graph (Weighted)           | Warehouses, delivery hubs, and customer locations are modeled as vertices with road distances as weighted edges for route planning.                                                                                                                                                  |

Implemented Feature (Top-Selling Priority Queue & Cart Deque) & Assert Tests:

```
Lab 11 - Task 8: Smart E-Commerce Platform Data Structure Architecture
Student: P HARSHAVARDHAN (Roll No: 2503A52366, Batch: 13)

import heapq
from typing import List, Dict, Tuple, Any, Optional

class TopSellingProductTracker:
    """
    Implements a Priority Queue (Max-Heap via inverted sales count)
    to dynamically track and rank top-selling e-commerce items in O(log N) time.
    """
    def __init__(self) -> None:
        self._heap: List[Tuple[int, str, float]] = [] # (-sales_count, product_name, price)
        self._product_map: Dict[str, int] = {} # product_name -> total sales

    def record_sale(self, product_name: str, quantity: int, unit_price: float) -> None:
        """Records product sales and updates dynamic sales volume."""
        self._product_map[product_name] = self._product_map.get(product_name, 0) + quantity
        total_sales = self._product_map[product_name]
        # Invert sales count for max-heap behavior
        heapq.heappush(self._heap, (-total_sales, product_name, unit_price))

    def get_top_k_products(self, k: int = 3) -> List[Dict[str, Any]]:
        """
        Extracts the top K bestselling products.
        """
        # Deduplicate to get the latest highest sales record per product
        seen = set()
        top_list = []
        # Sort current heap snapshot
        sorted_snapshot = sorted(self._heap)
        for neg_sales, name, price in sorted_snapshot:
            if name not in seen:
                seen.add(name)
                top_list.append({
                    "rank": len(top_list) + 1,
                    "product": name,
                    "units_sold": -neg_sales,
                    "unit_price": price,
                    "total_revenue": round(-neg_sales * price, 2)
                })
            if len(top_list) == k:
                break
        return top_list

class ShoppingCartManager:
    """
    Implements a Deque/List for dynamic shopping cart item additions, removals, and checkout.
    """
    def __init__(self) -> None:
        self._items: List[Dict[str, Any]] = []

    def add_item(self, product_id: str, name: str, price: float, quantity: int = 1) -> None:
        self._items.append({"id": product_id, "name": name, "price": price, "qty": quantity})

    def remove_item(self, product_id: str) -> bool:
        for i, itm in enumerate(self._items):
            if itm["id"] == product_id:
                self._items.pop(i)
                return True
        return False

    def calculate_cart_total(self) -> float:
        return sum(itm["price"] * itm["qty"] for itm in self._items)

    def item_count(self) -> int:
        return len(self._items)

def test_ecommerce_platform():
    print(""" Running Test Assertions for Task 8 (Smart E-Commerce Platform Architecture) """)

    # Test 1: Top Selling Product Tracker (Priority Queue / Max-Heap)
```

```
tracker = TopSellingProductTracker()
tracker.record_sale("Wireless Headphones", 150, 49.99)
tracker.record_sale("Mechanical Keyboard", 300, 89.99)
tracker.record_sale("Smart Watch", 450, 199.99)
tracker.record_sale("USB-C Hub", 100, 24.99)

top_3 = tracker.get_top_k_products(k=3)
assert len(top_3) == 3, "Test 1 Failed"
assert top_3[0]["product"] == "Smart Watch" and top_3[0]["units_sold"] == 450, "Rank 1 mismatch"
assert top_3[1]["product"] == "Mechanical Keyboard" and top_3[1]["units_sold"] == 300, "Rank 2 mismatch"
assert top_3[2]["product"] == "Wireless Headphones" and top_3[2]["units_sold"] == 150, "Rank 3 mismatch"
print(f"Assertion 1 Passed: Top product correctly identified -> #{top_3[0]['rank']}: {top_3[0]['product']} ({top_3[0]['units_sold']} units sold)")

# Test 2: Dynamic Sales Update
tracker.record_sale("Wireless Headphones", 400, 49.99) # Total: 550 (Surpasses Smart Watch)
new_top = tracker.get_top_k_products(k=1)
assert new_top[0]["product"] == "Wireless Headphones" and new_top[0]["units_sold"] == 550, "Rank update failed"
print(f"Assertion 2 Passed: Dynamic sale update propelled '{new_top[0]['product']}' to Rank #1 ({new_top[0]['units_sold']} units).")

# Test 3: Shopping Cart Management
cart = ShoppingCartManager()
cart.add_item("P101", "Laptop Stand", 29.99, 1)
cart.add_item("P102", "Wireless Mouse", 19.99, 2)
assert cart.item_count() == 2, "Test 3 Failed"
assert cart.calculate_cart_total() == round(29.99 + (19.99 * 2), 2), "Test 3 Total Mismatch"
print(f"Assertion 3 Passed: Shopping cart total calculated accurately -> Rs. {cart.calculate_cart_total()}")

# Test 4: Cart item removal
assert cart.remove_item("P101") is True, "Remove failed"
assert cart.item_count() == 1, "Item count mismatch"
assert cart.remove_item("P999") is False, "Non-existent remove failed"
print("Assertion 4 Passed: Item removed from shopping cart dynamically.")

print("All 4 Task 8 Assertions passed successfully!")

if __name__ == "__main__":
    test_ecommerce_platform()
```

Output (VS Code Integrated Terminal):



Comparison Analysis Table:

| Architecture Aspect   | Monolithic Array / List Approach                                                                                                    | Specialized Multi-Structure Architecture                                                                                                   |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Bestseller Rankings   |                                                                                                                                     |                                                                                                                                            |
| Product Search        | Sorting entire catalog on every sale costs $O(N \log N)$ CPU time. Linear scan $O(N)$ causes severe search latency on big catalogs. | Max-Heap updates rankings dynamically in $O(\log N)$ time.                                                                                 |
| Cart Management       |                                                                                                                                     | Hash Table achieves sub-millisecond $O(1)$ direct ID lookup.                                                                               |
| High-Traffic Handling | Array shifts on removing cart items cause memory churn.<br>System crashes under heavy Black Friday concurrent traffic.              | Deque provides $O(1)$ front and rear item additions and removals.<br>Decoupled specialized data structures scale smoothly under peak load. |